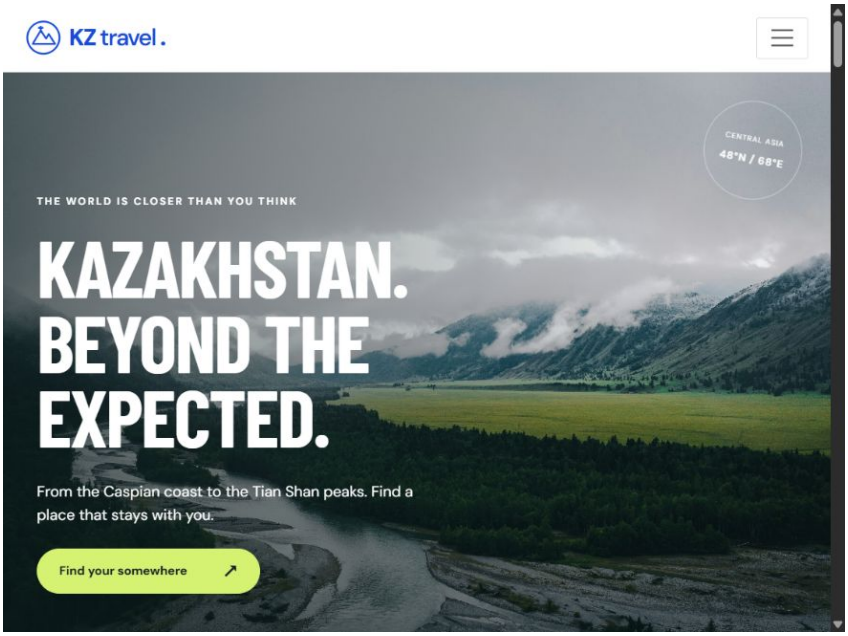
Go a little further. Assignment 5

Advanced JavaScript and DOM Manipulation



KZ Travel: the published Kazakhstan guide using the original project photographs.

Objective. Continue the six-page travel project with an engaging interface that demonstrates JavaScript fundamentals, DOM manipulation, events, callbacks, data structures, sound and animations.

Team	KZ Travel
Members	Ramazan Utegen Ziyadinkhan Kudaibergenuly Ruzimuhammad
Group	SE-2501

Publication: Updated deployment verified 2026-10-23.

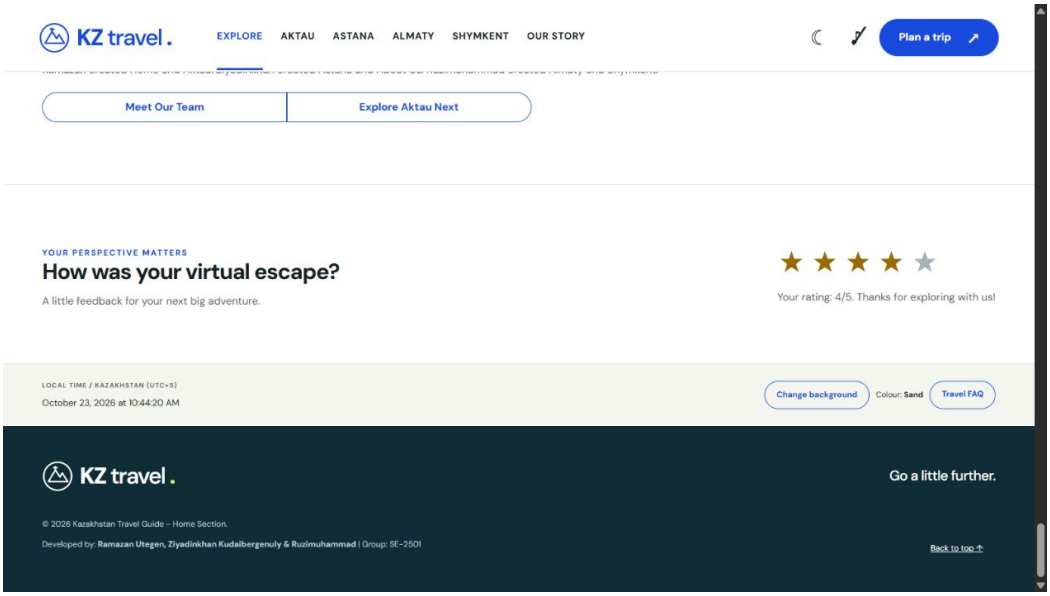
Deployed website: https://ramazanutegen.github.io/assignment1web/

Verified website source: 80fff147d854; GitHub Pages build and deployment completed successfully.

Select, update and rate.

Part 1.1: DOM selection, dynamic text, class-based styling and the chosen rating-star option for attribute manipulation.

1. Select the rating group and its five stars with querySelector/querySelectorAll.
2. A click updates the selected value, CSS class, aria-pressed and textContent feedback.
3. Store the page-specific rating locally; reload to restore it. User text is never inserted as HTML.



Result in the local browser. Original captures are included in docs/assignment-5/screenshots/.

KZ TRAVEL / ASSIGNMENT 5 / SOURCE CAPTURE

DOM selection and star-rating updates

js/experience.js

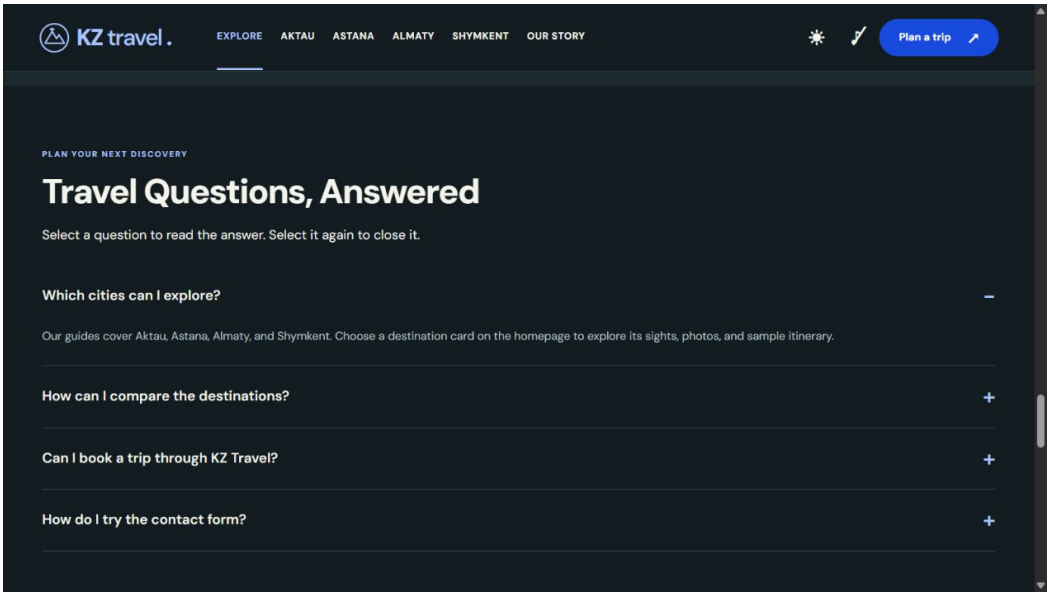
```
218     document.querySelectorAll('[data-rating]').forEach(rating => {
219         const stars = rating.querySelectorAll('[data-star]');
220         const message = rating.querySelector('[data-rating-message]');
221         const key = 'rating-' + rating.dataset.rating;
222         let selected = Number(storage.read(key, 0));
223         if (!Number.isInteger(selected) || selected < 0 || selected > 5) selected = 0;
224         function renderRating() {
225             stars.forEach(star => {
226                 star.classList.toggle('is-selected', Number(star.dataset.star) <= selected);
227                 star.setAttribute('aria-pressed', String(Number(star.dataset.star) === selected));
228             });
229             message.textContent = selected ? `Your rating: ${selected}/5. Thanks for exploring with us!` :
    'Choose your rating';
230         }
231         stars.forEach(star => star.addEventListener('click', () => {
232             selected = Number(star.dataset.star);
233             storage.write(key, selected);
234             renderRating();
235             animate(message);
236             playChime();
237         }));
238         renderRating();
239     });
240
```

Source-code screenshot, rendered from the actual project file with original line numbers.

A guide for day and night.

Part 1.1: dynamic styling; chosen option: functional Day/Night switch.

1. The moon/sun button calls applyTheme and changes the body data-theme attribute.
2. CSS variables recolour surfaces, text, borders and controls across all six pages.
3. Persist the theme locally. Keep the earlier Sand/Sky/Steppe background cycle working with matching night colours.



Result in the local browser. Original captures are included in docs/assignment-5/screenshots/.

KZ TRAVEL / ASSIGNMENT 5 / SOURCE CAPTURE

Persistent day and night themes

js/experience.js

```
107     function applyTheme(theme) {
108         document.body.dataset.theme = theme;
109         themeToggle?.setAttribute('aria-pressed', String(theme === 'night'));
110         themeToggle?.setAttribute('aria-label', theme === 'night' ? 'Switch to day theme' : 'Switch to night
    theme');
111         if (themeToggle) themeToggle.textContent = theme === 'night' ? '☀' : '☾';
112         document.dispatchEvent(new Event('kz-theme-change'));
113     }
114     applyTheme(storage.read('theme', 'day') === 'night' ? 'night' : 'day');
115     themeToggle?.addEventListener('click', () => {
116         const theme = document.body.dataset.theme === 'night' ? 'day' : 'night';
117         applyTheme(theme);
118         storage.write('theme', theme);
119     });
120
121     // Arrow keys move focus through the real navigation links without trapping Tab.
```

Source-code screenshot, rendered from the actual project file with original line numbers.

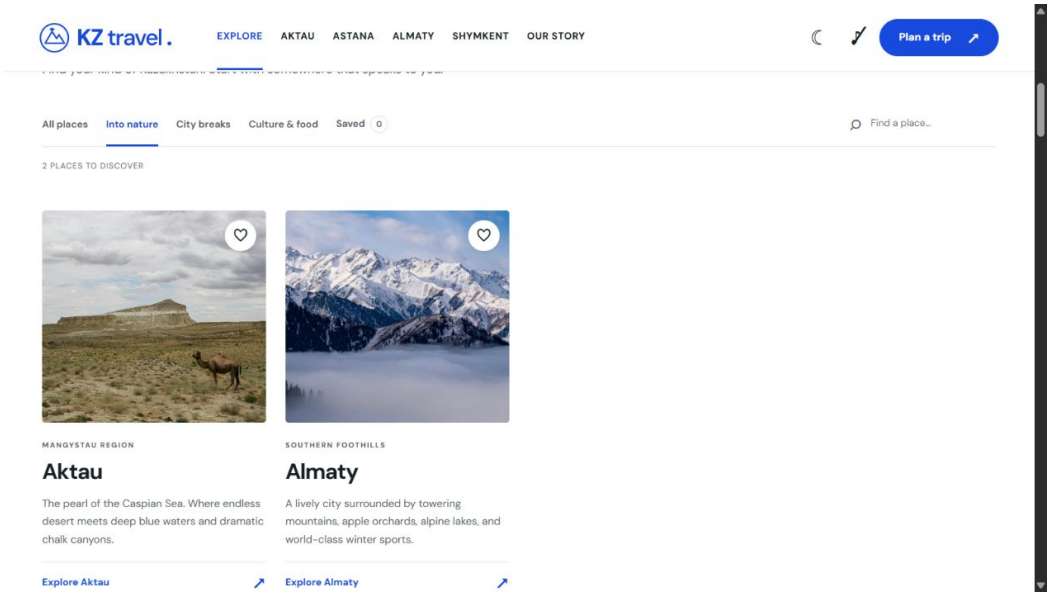
Make discovery respond.

Part 1.2: addEventListener, keyboard handling and switch statements.

1. Bind click/input listeners to category buttons, saved hearts and destination search.

2. Use a switch to choose category logic; combine it with the search query and update results without reload.

3. Use keydown and a switch for four arrow keys, Home and End. Move focus through navigation links and preserve normal Tab behaviour.



Result in the local browser. Original captures are included in docs/assignment-5/screenshots/.

KZ TRAVEL / ASSIGNMENT 5 / SOURCE CAPTURE

Keyboard listeners and switch logic

js/experience.js

```
122     const menuLinks = Array.from(document.querySelectorAll('#main-navigation .nav-link'));
123     for (let index = 0; index < menuLinks.length; index += 1) {
124         menuLinks[index].addEventListener('keydown', event => {
125             let nextIndex;
126             switch (event.key) {
127                 case 'ArrowRight': case 'ArrowDown': nextIndex = (index + 1) % menuLinks.length; break;
128                 case 'ArrowLeft': case 'ArrowUp': nextIndex = (index - 1 + menuLinks.length) %
    menuLinks.length; break;
129                 case 'Home': nextIndex = 0; break;
130                 case 'End': nextIndex = menuLinks.length - 1; break;
131                 default: return;
132             }
133             event.preventDefault();
134             menuLinks[nextIndex].focus();
135         });
136     }
137
```

Source-code screenshot, rendered from the actual project file with original line numbers.

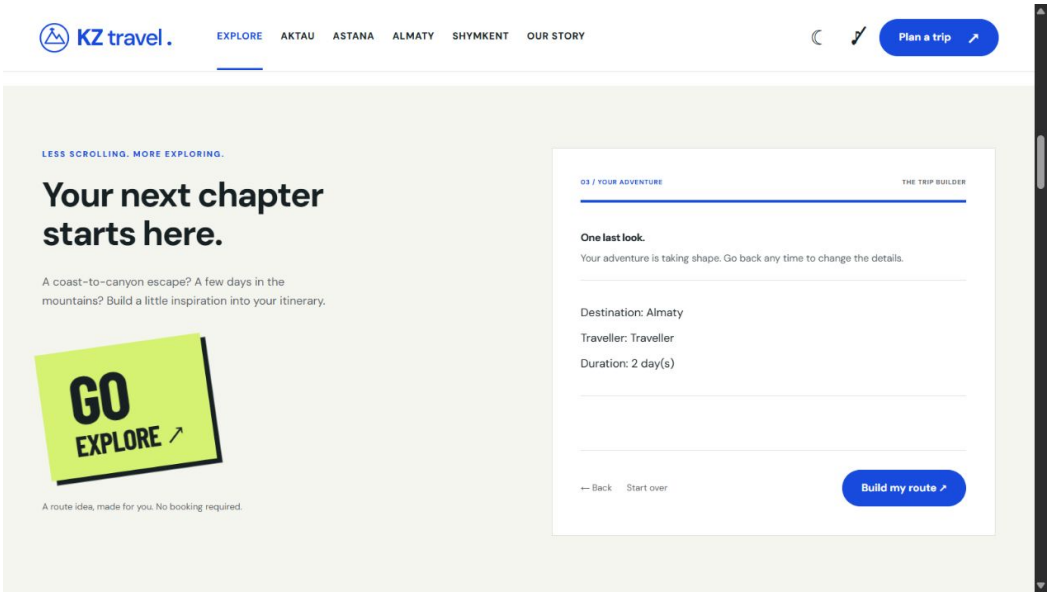
One journey, three steps.

Part 1.2: chosen callback option: multi-step form with Next and Back.

1. Create destination, traveller-details and review fieldsets. Hidden steps remain in the DOM.

2. changeStep validates the current step, changes its index, then invokes the onStepChanged callback.

3. Pass showStep as that callback. Back/Next preserve inputs; missing choices show feedback; Start over resets the form.



Result in the local browser. Original captures are included in docs/assignment-5/screenshots/.

KZ TRAVEL / ASSIGNMENT 5 / SOURCE CAPTURE

Multi-step form: validation and rendering callback

js/experience.js

```
279         function changeStep(direction, onStepChanged) {
280             if (direction > 0 && step === 0 && !chosenDestination()) {
281                 error.textContent = 'Choose a destination to start your journey.';
282                 planner.querySelector('[name=destination]').focus();
283                 return;
284             }
285             if (direction > 0 && step === 1 && !planner.elements.traveler.value.trim()) {
286                 error.textContent = 'Enter your first name before continuing.';
287                 planner.elements.traveler.focus();
288                 return;
289             }
290             step = Math.max(0, Math.min(2, step + direction));
291             onStepChanged(step);
292             steps[step].querySelector('input,select')?.focus();
293         }
```

Source-code screenshot, rendered from the actual project file with original line numbers.

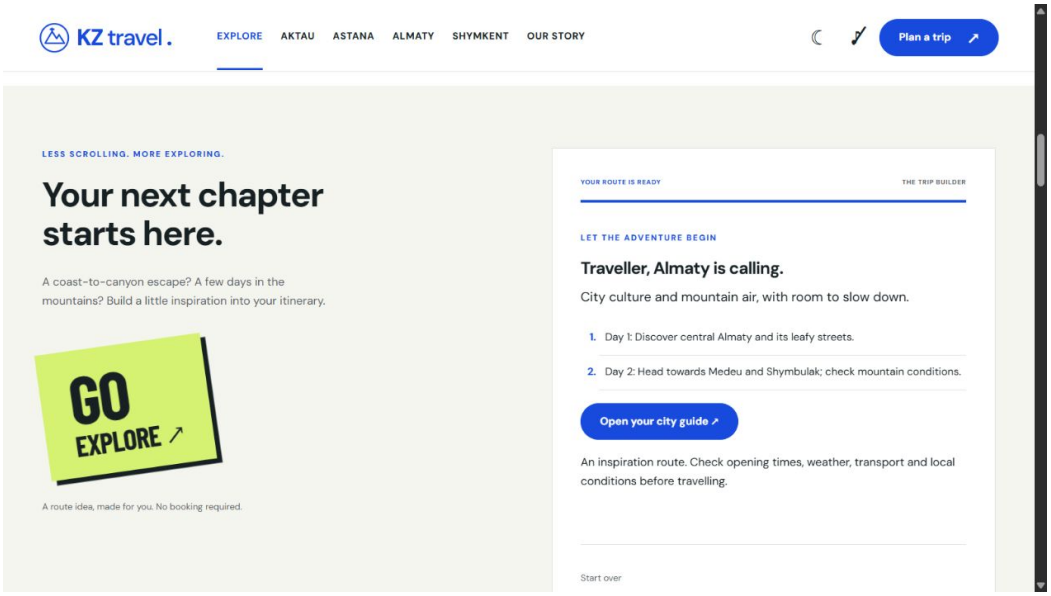
Turn data into a route.

Part 1.3: objects and methods, arrays, loops and higher-order functions.

1. Represent each city as an object containing its name, category, description and stops array.

2. Add getRoute(days), which returns this.stops.slice(0, days). Use map/filter/forEach to process collections.

3. Build the selected route with an explicit for loop, createElement and textContent. A two-day choice generates exactly two day items.



Result in the local browser. Original captures are included in docs/assignment-5/screenshots/.

KZ TRAVEL / ASSIGNMENT 5 / SOURCE CAPTURE

Objects, methods, higher-order functions and a loop

js/experience.js

```
18      const destinations = [
19          { id: 'aktau', name: 'Aktau', category: 'nature', tags: 'sea coast desert canyons mangystau',
20              description: 'A Caspian escape, with wide horizons and dramatic landscapes.',
21              stops: ['Walk the Caspian waterfront and discover Aktau.', 'Explore Mangystau chalk formations
    with a local guide.', 'Make time for the Bozjyra landscapes and the open steppe.'] },
22          { id: 'astana', name: 'Astana', category: 'city', tags: 'capital architecture city river',
23              description: 'A fresh perspective on architecture, art and life on the steppe.',
24              stops: ['Explore the Baiterek area and Nurzhol Boulevard.', 'Discover Khan Shatyr and the city
    architecture.', 'Take a slower walk beside the Ishim River.'] },
```

js/experience.js

```
32          ...destination,
33          getRoute(days) { return this.stops.slice(0, days); }
34      }));
```

js/experience.js

```
299             const stops = planner.querySelector('[data-trip-stops]');
300             stops.replaceChildren();
301             // Explicit loop demonstrates array iteration while generating actual DOM nodes.
302             const route = place.getRoute(days);
303             for (let index = 0; index < route.length; index += 1) addText(stops, 'li', `Day ${index + 1}:
    ${route[index]}`);
```

Source-code screenshot, rendered from the actual project file with original line numbers.

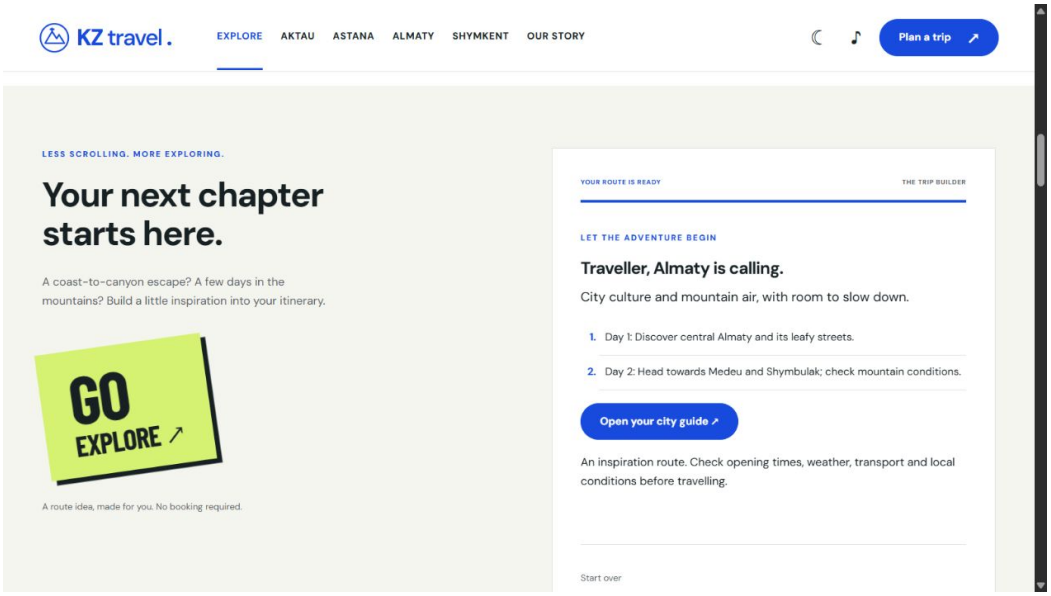
A small sound of discovery.

Part 1.3: JavaScript-triggered sound effects, controlled by the user.

1. Sound starts disabled. Clicking the music note enables audio and previews the chime.

2. A sine oscillator changes from 660 to 880 Hz. A short gain envelope fades the note out.

3. Saving, rating or completing a route can trigger it. A new sound stops the old oscillator; onended disconnects nodes. The toggle disables further sounds.



Result in the local browser. Original captures are included in docs/assignment-5/screenshots/.

KZ TRAVEL / ASSIGNMENT 5 / SOURCE CAPTURE

A short, user-triggered Web Audio chime

js/experience.js

```
75            const oscillator = audioContext.createOscillator();
76            const gain = audioContext.createGain();
77            const now = audioContext.currentTime;
78            oscillator.type = 'sine';
79            oscillator.frequency.setValueAtTime(660, now);
80            oscillator.frequency.setValueAtTime(880, now + 0.09);
81            gain.gain.setValueAtTime(0.0001, now);
82            gain.gain.exponentialRampToValueAtTime(0.12, now + 0.015);
83            gain.gain.exponentialRampToValueAtTime(0.0001, now + 0.23);
84            oscillator.connect(gain);
85            gain.connect(audioContext.destination);
86            activeSound = oscillator;
87            oscillator.onended = () => {
88                oscillator.disconnect();
89                gain.disconnect();
90                if (activeSound === oscillator) activeSound = null;
91            };
92            oscillator.start(now);
93            oscillator.stop(now + 0.24);
94        } catch {
95            notify('Your browser could not play the sound.');
96        }
```

Source-code screenshot, rendered from the actual project file with original line numbers.

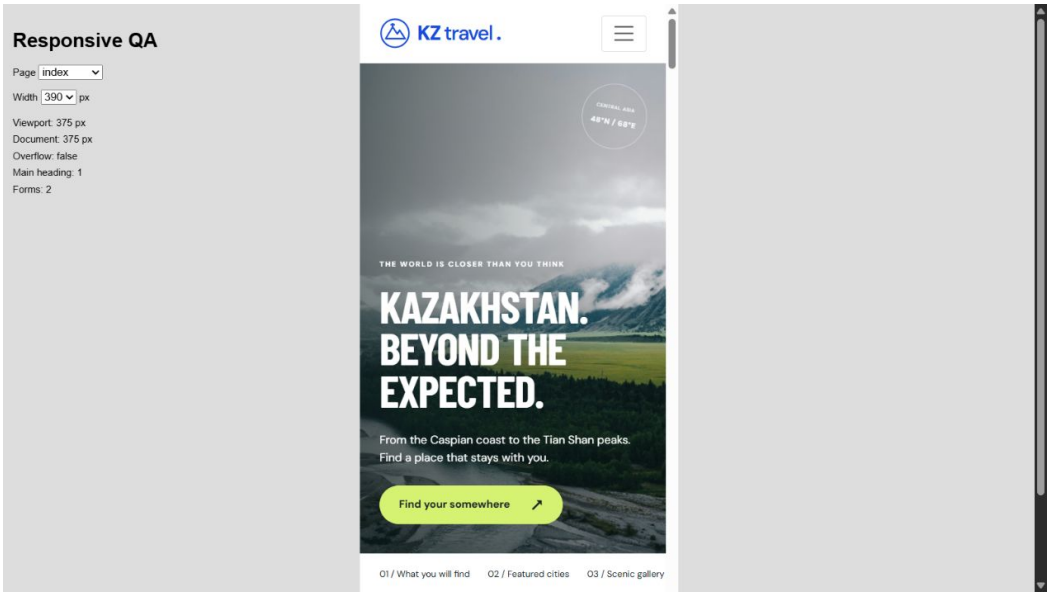
Motion with a purpose.

Parts 1.3-1.5: event-triggered animation, separation of concerns, clean structure and theme integration.

1. Restart ui-enter after filtering, rating or changing a step; use a pulse for a saved heart.

2. CSS controls transitions, keyframes and progress animation; prefers-reduced-motion disables movement.

3. Keep markup in HTML, presentation in editorial.css, and behaviour in experience.js. Use meaningful helper functions, local assets, responsive layouts and travel-specific content.



Result in the local browser. Original captures are included in docs/assignment-5/screenshots/.

KZ TRAVEL / ASSIGNMENT 5 / SOURCE CAPTURE

Event-triggered movement with reduced-motion support

js/experience.js

```
36      function animate(element) {
37          if (!element) return;
38          element.classList.remove('ui-enter');
39          // Restart the event-triggered CSS animation after the DOM update.
40          void element.offsetWidth;
41          element.classList.add('ui-enter');
42      }
43
```

css/editorial.css

```
1790 @media(prefers-reduced-motion:reduce) {
1791      html {
1792          scroll-behavior: auto
1793      }
1794
1795      .ui-enter,
1796      .save-place.is-pulsing {
1797          animation: none
1798      }
1799
1800      .editorial *,
```

Source-code screenshot, rendered from the actual project file with original line numbers.

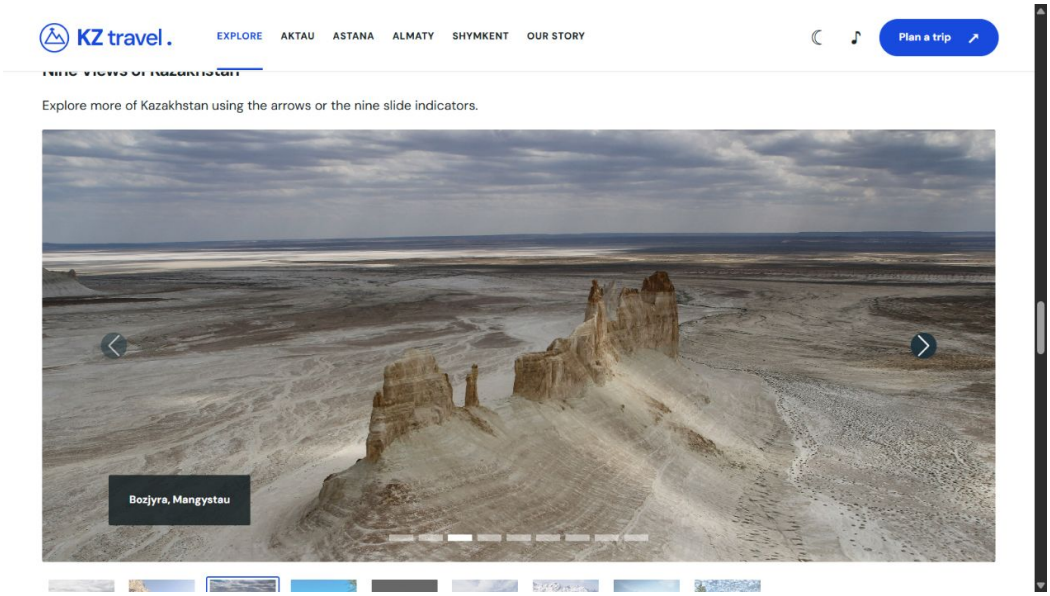
Keep every view of Kazakhstan.

Additional DOM enhancement: clickable gallery thumbnails while retaining the original manual galleries.

1. Retain the five nine-image Bootstrap carousels, original image sources, arrows and indicators.

2. Add a button for every thumbnail, targeting the matching slide through Bootstrap data attributes.

3. On slid.bs.carousel, update the thumbnail class and aria-pressed. Manual, thumbnail and keyboard navigation remain in sync.



Result in the local browser. Original captures are included in docs/assignment-5/screenshots/.

KZ TRAVEL / ASSIGNMENT 5 / SOURCE CAPTURE

Existing carousel and new thumbnail state

js/experience.js

```
208    document.querySelectorAll('[data-gallery-for]').forEach(thumbnails => {
209        const carousel = document.getElementById(thumbnails.dataset.galleryFor);
210        carousel?.addEventListener('slid.bs.carousel', event => {
211            thumbnails.querySelectorAll('button').forEach((button, index) => {
212                button.classList.toggle('is-active', index === event.to);
213                button.setAttribute('aria-pressed', String(index === event.to));
214            });
215        });
216    });
217
218    document.querySelectorAll('[data-rating]').forEach(rating => {
```

Source-code screenshot, rendered from the actual project file with original line numbers.

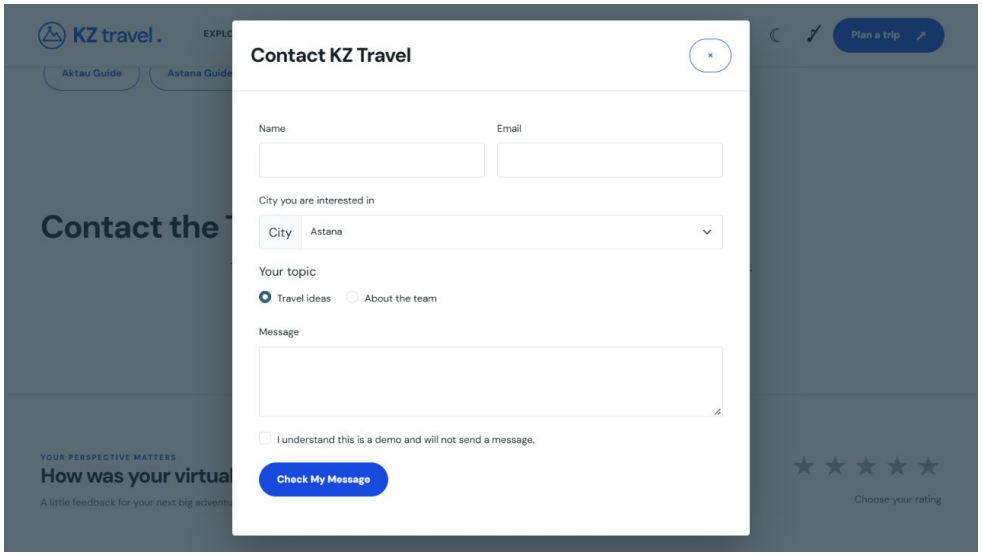
Earlier work, still working.

The redesign preserves the existing theme, all original image files and references, city content, sample routes, tables and team credits. The shared Bootstrap grid and the earlier CSS-only media-query card exercise remain.

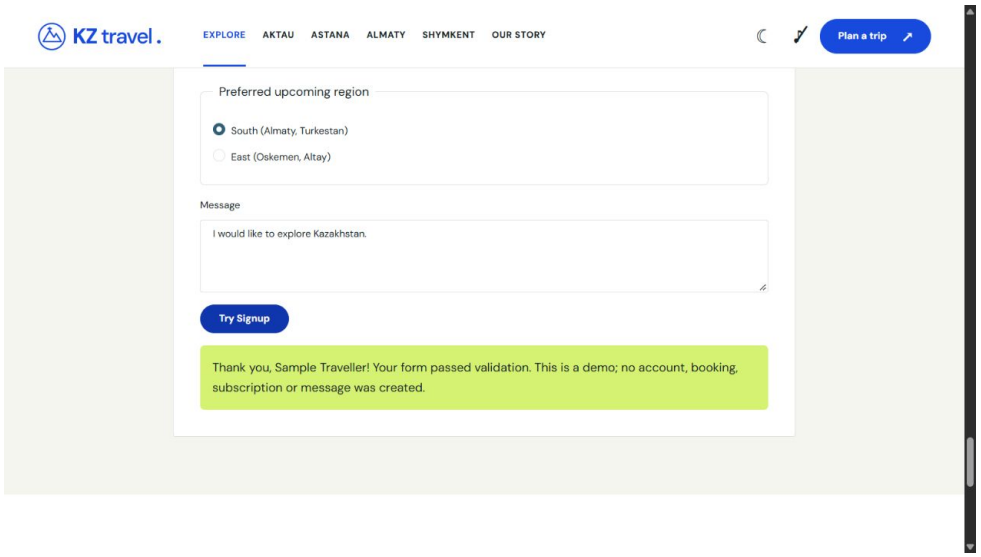
1. Legacy forms still validate required fields, email, password length and confirmation; successful validation stays on the page.

2. The homepage FAQ opens/closes; the About Us dialog closes with Escape and returns focus to Contact Us.

3. The clock updates in Asia/Almaty (UTC+5). Background cycling now appears in the footer.



Retained contact dialog; the screenshot shows the original form fields in the new design.



A valid sample form displays confirmation without transmitting data or creating a subscription.

Verification, reflection and publication.

Review environment: in-app Chromium on Windows, 1280 x 720 desktop viewport; same-origin responsive frames at 360, 390 and 768 CSS px. Frame inner widths exclude scrollbars. These checks do not claim physical-device or Firefox coverage.

Published-site recheck: Chromium at 896 x 668. All 54 checked HTML/assets returned HTTP 200 and matched the project. All six pages fitted the viewport; live interaction checks passed and no console errors were observed.

Check	Observed result
Six desktop pages	No horizontal overflow or broken loaded images
18 responsive layouts	All six pages fit 360 / 390 / 768 px frames
Search, filters and saved places	Combined criteria, empty state and persistence work
Planner and callbacks	Validation, Back, review, correct route length and reset
Theme, rating and navigation	Persistence, star feedback and arrow-key focus work
Legacy functions	Form validation, FAQ, galleries, popup and focus return
Static source check	Resources, anchors, IDs, credits and all original photos

Challenges and solutions. Existing features survived by retaining their scripts and changing the shared presentation layer. Theme variables resolve day/night consistency with the older background control. Hiding fieldsets preserves input on Back. textContent safely displays names; local dependency files allow offline use. Sound starts only after interaction and its audio nodes are cleaned up.

Reflection and defence. Each member should trace an event from its listener to its callback, data processing, DOM update and CSS effect. Practise explaining this, getRoute, switch, filter/map/forEach, the explicit for loop, aria-pressed, hidden, localStorage and reduced-motion handling. Rehearsal notes are included in English and Russian.

Part 2 - updated website deployment. The redesign was committed and pushed to main. GitHub Pages deployed source 80fff147d854. The published site was verified on 2026-10-23: all six pages and their local assets were checked, and live browser tests covered discovery, saved places, themes, ratings, keyboard navigation, the planner, galleries, FAQ and the contact dialog.

https://ramazanutegen.github.io/assignment1web/

Deployment evidence: successful GitHub Pages workflow; detailed live checks and asset hashes are in docs/assignment-5/deployment.json.

Submission package: the approved site is published; the PDF and ZIP are updated. Every team member submits both files and attends the practice-lesson defence. LMS deadline supplied by the team: 29 October 2026, 14:00; follow the LMS time zone.